

A
Major Project Stage-I Report On
Eye Movement Controlled Cursor Using OpenCV
A Hands-Free Mouse Solution
(Submitted partial fulfillment of the requirements for the award of Degree)
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING (Cyber Security)

By

SUCHITHRA RAVULA	227R1A6249
SWETHA THADEM	227R1A6258
MURALI KRISHNA VANGA	227R1A6261
FARHAN ABASSI	227R1A6232

Under the Guidance of

Mrs. G.V.Ashritha

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)
Recognized Under Section 2(f) & 12(B) of the UGC Act, 1956, Kandlakoya (V), Medchal Road,
Hyderabad-501401.

November 2025.



CERTIFICATE

This is to certify that the project entitled “**Eye Movement Controlled Cursor Using Open CV -A Hands Free Mouse Solution**”being submitted by

Suchithra Ravula(227R1A6249),Swetha Thadem(227R1A6258),Murali krishna Vanga (227R1A6261),Farhaan Abassi(227R1A6232)in partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-2026.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs.G.V.Ashritha
Assistant Professor
INTERNAL GUIDE

Dr. K. Murali
HoD-CSE(DS)

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mrs.G.V.Ashritha, Assistant** for **her** exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by **her** shall carry us a long way in the journey of life on which we are about to embark.

We also take this opportunity to express a deep sense of gratitude to the Project Review Committee (PRC) coordinators **V. Tejaswini, J. Shiva** for their cordial support, valuable information and guidance, which helped us in completing this task through various stages.

We are also thankful to **Dr. K. Murali**, Head, Department of Computer Science and Engineering (Data Science) for providing encouragement and support for completing this project successfully.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to **Sri. Ch. Gopal Reddy**, Chairman, **Smt. C. Vasantha Latha**, Secretary and **Sri. C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

The guidance and support received from all the members of CMR Technical Campus who contributed to the completion of the project. We are grateful for their constant support and help.

Finally, we would like to take this opportunity to thank our family for their constant encouragement, without which this assignment would not be completed. We sincerely acknowledge and thank all those who gave support directly and indirectly in the completion of this project.

SUCHITHRA RAVULA	227R1A6249
SWETHA THADEM	227R1A6258
MURALI KRISHNA VANGA	227R1A6261
FARHAN ABASSI	227R1A6232

ABSTRACT

In the modern era of human-computer interaction, accessibility and automation have become essential components of technology-driven solutions. The project **“Eye Movement Controlled Cursor Using OpenCV – A Hands-Free Mouse Solution”** aims to design and develop a vision-based system that enables users to control the movement of a computer cursor using their eye movements, eliminating the need for traditional hardware devices such as a mouse or touchpad. This system leverages **computer vision techniques** and **machine learning algorithms** through the **OpenCV library** to capture and analyze real-time video frames from a webcam. It detects facial landmarks, identifies the eyes, and tracks the movement of the pupils to determine gaze direction. The detected gaze coordinates are then mapped to corresponding cursor movements on the screen. Additionally, **blink detection** is implemented to simulate mouse click events, allowing complete control of the system through natural eye gestures. This work aims to show the process of controlling a computer cursor by eye movement. A number of modules were created to interface the digital and physical worlds already available. We present a novel approach to human-computer interaction using a real-time camera and color pointers as opposed to the conventional cursor control. Using camera and computer vision technology such as image segmentation, background subtraction, and color tracking, we demonstrate the capability of a computer mouse to perform these tasks like current mouse devices.

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	v
1. INTRODUCTION	1
1.1 PROJECT PURPOSE	1
1.2 PROJECT FEATURES	2
2. LITERATURE SURVEY	3
2.1 REVIEW OF RELATED WORK	7
2.2 DEFINITION OF PROBLEM STATEMENT	9
2.3 EXISTING SYSTEM	9
2.4 PROPOSED SYSTEM	11
2.5 OBJECTIVES	12
2.6 HARDWARE & SOFTWARE REQUIREMENTS	13
2.6.1 HARDWARE REQUIREMENTS	13
2.6.2 SOFTWARE REQUIREMENTS	13
3. SYSTEM ARCHITECTURE & DESIGN	14
3.1 PROJECT ARCHITECTURE	14
3.2 DESCRIPTION	15
3.3 DATA FLOW DIAGRAM	16
4. IMPLEMENTATION	18
4.1 ALGORITHMS USED	18
4.2 SAMPLE CODE	25
5. RESULTS & DISCUSSION	37
6. VALIDATION	47
6.1 INTRODUCTION	47
6.2 TEST CASES	48
6.2.1 UPLOADING DATASET	48
6.2.2 CLASSIFICATION	48
7. CONCLUSION & FUTURE ASPECTS	49
7.1 PROJECT CONCLUSION	49
7.2 FUTURE ASPECTS	50
8. BIBLIOGRAPHY	51
8.1 REFERENCES	51
8.2 GITHUB LINK	52

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos.	14
Figure 3.2	Dataflow Diagram of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos.	17
Figure 4.1	Dataset directory structure with folders 'Safe Content and Inappropriate Content' having all the training examples.	23
Figure 4.2	Screenshot of the "Inappropriate Content" Folder Showing Sample Image Files	23
Figure 4.3	Screenshot of the "Safe Content" Folder Showing Sample Image Files.	24
Figure 5.1	GUI/Main Interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	37
Figure 5.2	Loaded sample image of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	38
Figure 5.3	Dataset of A Deep Learning- Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	39
Figure 5.4	Statistical Representation of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	40
Figure 5.5	Display of no. of items and labels present of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	41
Figure 5.6	Evaluation Metrics and Confusion Matrix of the proposed EfficientNet-BiLSTM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	42
Figure 5.7	Evaluation Metrics and Confusion Matrix of the EfficientNet-SVM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	43

Figure 5.8	Evaluation Metrics and Confusion Matrix of the EfficientNet-RF Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	44
Figure 5.9	Comparison graph of all three models of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	45
Figure 5.10	“Safe Content” prediction result displayed in the system interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	46

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Dataset	48
Table 6.2.2	Classification	48

1. INTRODUCTION

1. INTRODUCTION

In today's digital era, computers have become an integral part of daily life, serving as essential tools for communication, education, business, and entertainment. The traditional methods of interacting with computers, such as using a keyboard or a mouse, are effective but have limitations—especially for individuals who suffer from motor impairments or physical disabilities. As technology progresses, there is a growing need for **alternative and assistive interaction methods** that provide intuitive, hands-free control of digital devices.

The project titled “**Eye Movement Controlled Cursor Using OpenCV – A Hands-Free Mouse Solution**” introduces a novel approach to **human-computer interaction (HCI)** by replacing physical input devices with an intelligent computer vision-based system. This system allows users to control the movement of the cursor using **eye gaze direction** and execute click operations using **blink gestures**. The project utilizes **OpenCV**, a widely used open-source computer vision library in Python, to process real-time webcam input and detect eye movement patterns.

The core idea of this project is to create a **hands-free and accessible computing experience** that empowers users, particularly those with limited mobility, to operate a computer without physical effort. By integrating **face and eye detection algorithms**, **pupil tracking**, and **blink recognition**, the system converts natural eye movements into mouse control commands, enabling a seamless, real-time, and efficient interaction between humans and computers.

This project also highlights the importance of **assistive technologies** in promoting inclusivity and accessibility. Unlike existing hardware-based eye trackers, which are costly and require calibration or specialized sensors, this model uses a simple webcam, making it a **cost-effective and portable solution**. It demonstrates how computer vision and automation can come together to enhance usability and redefine the boundaries of digital interaction.

1.1 PROJECT PURPOSE

The primary purpose of this project is to design and develop an **eye-controlled computer cursor system** that operates solely based on the user's eye movements and blinks. The system aims to eliminate the dependency on physical input devices like a

mouse, making it particularly beneficial for **differently-abled individuals** who face challenges using traditional computer peripherals.

Key goals of the project include:

- To enable computer control through real-time eye tracking.
- To provide a **hands-free computing interface** that increases accessibility and independence.
- To implement an **efficient and low-cost alternative** to commercial eye-tracking systems.
- To demonstrate the practical application of **computer vision and machine learning techniques** in assistive technology.
- To improve user comfort and reduce fatigue by allowing natural, gaze-based interaction with digital environments.

Ultimately, the purpose of this system is to combine innovation with social impact by developing a tool that enhances inclusivity and empowers users through technology.

1.2 PROJECT FEATURES

The **Eye Movement Controlled Cursor System** offers several innovative and practical features that make it an effective human-computer interaction tool. These features are designed to ensure real-time responsiveness, accuracy, and ease of use.

Key Features:

Real-Time Eye Tracking: Uses OpenCV to continuously detect and track eye positions from a live webcam feed.

Pupil Detection and Gaze Estimation: Determines cursor direction based on the movement of the pupil across the frame.

Blink-Based Click Detection: Interprets eye blinks as mouse click events, replacing the need for manual button presses.

No External Hardware: Requires only a standard webcam—no infrared sensors or wearable devices.

Software-Based Implementation: Built entirely with Python and open-source libraries such as OpenCV, Dlib, and PyAutoGUI.

Customizable Sensitivity: Allows calibration to adjust to different users, lighting conditions, and camera resolutions.

Accessible and Inclusive Design: Designed to help physically challenged users operate computers independently.

Real-Time Performance: Efficient frame processing ensures smooth cursor movement with minimal delay.

This system's combination of simplicity, accuracy, and low cost makes it a promising solution for **assistive technology** and an important step toward natural, gaze-based human-computer interaction.

2. LITERATURE REVIEW

2. LITERATURE REVIEW

The development of **eye movement-controlled systems** has been an active research area in **human-computer interaction (HCI)**, **assistive technology**, and **computer vision**. The fundamental goal of such systems is to provide an alternative method of input that allows users to interact with computers and digital devices through **eye tracking**, **gaze estimation**, and **blink recognition**.

This section reviews key studies, existing systems, and technological developments relevant to the “**Eye Movement Controlled Cursor Using OpenCV – A Hands-Free Mouse Solution**” project. It also presents the **problem definition**, **proposed approach**, and the **objectives** that drive the system’s design.

2.1 REVIEW OF RELATED WORK

Several researchers and developers have explored **gaze-based control systems** and **vision-driven interaction models** over the past decade.

- **P. Majaranta and A. Bulling (2014)** in *Eye Tracking and Eye-Based Human–Computer Interaction* described how eye tracking technology evolved from medical research tools into mainstream assistive devices. They highlighted the potential of vision-based systems for enabling hands-free interaction without physical strain.
- **D. W. Hansen and Q. Ji (2010)** presented a survey of gaze estimation techniques, categorizing them into geometry-based, appearance-based, and machine learning-based methods. Their research demonstrated that accurate gaze detection is achievable even with low-cost cameras under controlled conditions.
- **A. Kumar and S. S. Rautaray (2018)** developed an eye-controlled mouse using OpenCV and Haar Cascades. Their system tracked real-time eye movement and mapped it to cursor control, establishing a foundation for software-only solutions in gaze-based computing.
- **S. Kherchaoui and A. Houacine (2016)** introduced a facial and eye detection system using the Viola–Jones algorithm, which proved effective for real-time applications due to its computational efficiency.
- **Y. Sugano et al. (2014)** applied convolutional neural networks (CNNs) to improve gaze estimation accuracy under varying head poses, marking a step forward for integrating deep learning in gaze-controlled systems.

From these studies, it is evident that real-time eye tracking has matured significantly. However, challenges such as **lighting sensitivity**, **head movement**, and **processing speed** remain open areas for research. The current project addresses these issues by optimizing face and eye detection using **OpenCV**, and simplifying user calibration for improved usability.

2.2 DEFINITION OF PROBLEM STATEMENT

Despite advancements in technology, traditional input devices like the mouse and keyboard still dominate human-computer interaction. These devices are unsuitable for individuals with **motor impairments**, **neuromuscular disorders**, or **injuries** that prevent physical control of hardware. Existing gaze-tracking devices often rely on **infrared sensors** or **specialized headgear**, which are expensive, uncomfortable, and impractical for everyday use. Therefore, there is a need for a **software-based, low-cost, real-time eye-tracking system** that allows cursor control and clicking without external devices.

Problem

To design and implement a computer vision-based system that uses real-time eye tracking and

Statement:

blink detection to control the computer cursor, thereby enabling a hands-free and accessible user experience for individuals with physical disabilities.

2.3 EXISTING SYSTEM

Existing eye-tracking systems and gaze-controlled interfaces are either **hardware-dependent** or **limited in precision** when implemented in software-only environments.

Limitations of Existing Systems:

- Require infrared cameras or special headgear for accurate gaze tracking.
- High setup cost and limited availability.
- Sensitive to lighting variations and camera quality.
- Complex calibration procedures.
- Not suitable for real-time consumer applications.

These constraints make such systems difficult to deploy widely, especially in assistive contexts.

2.4 PROPOSED SYSTEM

The proposed **Eye Movement Controlled Cursor System** offers a **cost-effective, portable, and software-based** alternative using **OpenCV** and a standard webcam.

The system uses the following workflow:

1. **Capture Input:** Webcam captures real-time video frames of the user's face.
2. **Detect Face and Eyes:** OpenCV's Haar Cascade Classifiers identify face and eye regions.
3. **Pupil Tracking:** The system detects and tracks pupil movement within the eye region.
4. **Gaze Estimation:** The displacement of the pupil is mapped to cursor motion on the screen using PyAutoGUI.
5. **Blink Detection:** A blink pattern (e.g., prolonged eye closure) triggers a click action.

Advantages of the Proposed System:

- No need for external sensors or expensive equipment.
- Fully software-driven and easily customizable.
- Suitable for real-time applications.
- Lightweight and platform-independent.
- Increases accessibility for physically challenged users.

The proposed model thus offers a scalable, user-friendly, and affordable alternative for cursor control.

2.5 OBJECTIVES

The key objectives of this project are:

- To design a **real-time eye tracking system** using OpenCV and a standard webcam.
- To implement **cursor control and clicking** based solely on eye movements and blinks.
- To minimize dependency on hardware-based devices for user interaction.
- To improve **accessibility and usability** for physically challenged users.
- To provide an **open-source and customizable framework** for further research and development in gaze-controlled systems.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 Hardware Requirements

- Webcam (minimum 720p recommended)
- Personal Computer or Laptop with i3 or above processor
- Minimum 4 GB RAM
- Stable lighting environment

2.6.2 Software Requirements

- **Programming Language:** Python
- **Libraries:** OpenCV, Dlib, NumPy, PyAutoGUI, imutils
- **Operating System:** Windows / Linux
- **IDE:** PyCharm or Visual Studio Code
- **Version Control:** Git / GitHub

3 SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

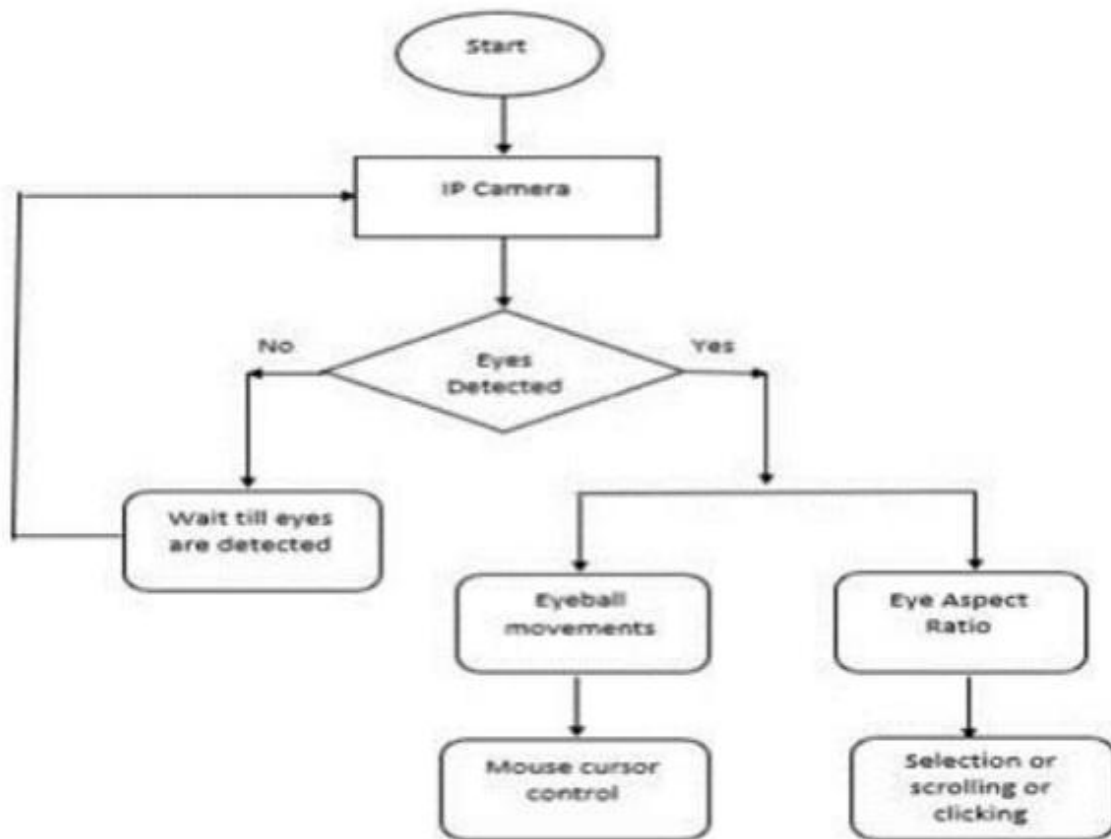
The **Eye Movement Controlled Cursor Using OpenCV – A Hands-Free Mouse Solution** follows a modular system design that ensures efficient real-time eye movement tracking and accurate cursor control. The architecture integrates image acquisition, preprocessing, detection, and control layers in a streamlined manner to achieve a seamless human-computer interaction experience.

3.1 PROJECT ARCHITECTURE

The system architecture defines how each functional module interacts to perform the overall task of cursor control through eye movements. It consists of several stages — from capturing the user's face and eyes to analyzing gaze direction and executing cursor actions.

Key Components:

- 1. User and Webcam Input:**
The process begins when the webcam captures continuous frames of the user's face and eyes. The camera acts as a sensor that supplies real-time video data for analysis.
- 2. Image Preprocessing:**
Each captured frame is converted into a grayscale image to reduce computational complexity. Noise reduction filters like **Gaussian Blur** or **Median Blur** are applied to enhance the visibility of edges and features.
- 3. Face and Eye Detection:**
Using **OpenCV's Haar Cascade Classifier** or **Dlib's facial landmark detector**, the system locates the face within the image. Once detected, the **Region of Interest (ROI)** for the eyes is extracted for more focused processing.
- 4. Pupil Detection and Tracking:**
The pupil is identified as the darkest circular region in the eye. Thresholding and contour detection methods are applied to find the pupil's position. Its displacement from the eye's center indicates the direction of gaze.
- 5. Gaze Estimation (Direction Calculation):**
The relative movement of the pupil is analyzed to determine whether the user is looking left, right, up, down, or center. These movements correspond to cursor direction commands.
- 6. Blink Detection (Click Event):**
Blink detection is achieved through the **Eye Aspect Ratio (EAR)** method using Dlib landmarks. When the EAR drops below a defined threshold for a certain duration, it triggers a mouse click (single or double).
- 7. Cursor Control (Screen Movement):**
The **PyAutoGUI** library maps the detected gaze direction and blink signals to actual cursor movement and click events on the screen, providing hands-free control.



3.2 DESCRIPTION OF MODULES

Each component in the system performs a specific role to achieve accurate, real-time cursor control.

1. Image Acquisition Module

- Captures continuous video stream using a standard webcam.
- Provides 20–30 frames per second for smooth tracking.
- Ensures stable lighting and minimal head movement for best accuracy.

2. Image Preprocessing Module

- Converts RGB frames to grayscale for faster processing.
- Applies noise filters (Gaussian or Bilateral filters).
- Enhances contrast for accurate detection under variable lighting conditions.

3. Face Detection Module

- Detects the user's face using **Haar Cascade Classifiers**.
- Determines bounding coordinates to extract facial ROI.
- Ensures robust detection even with slight head rotation.

4. Eye Detection Module

- Detects eyes within the facial ROI using either Haar cascades or Dlib landmarks.
- Defines precise coordinates of both eyes to isolate them for further processing.

5. Pupil Detection Module

- Applies binary thresholding to highlight dark pupil regions.
- Uses **contour detection** to locate pupil center coordinates.
- Tracks pupil movement across consecutive frames to identify gaze direction.

6. Gaze Estimation Module

- Determines gaze direction by comparing pupil position with eye center.

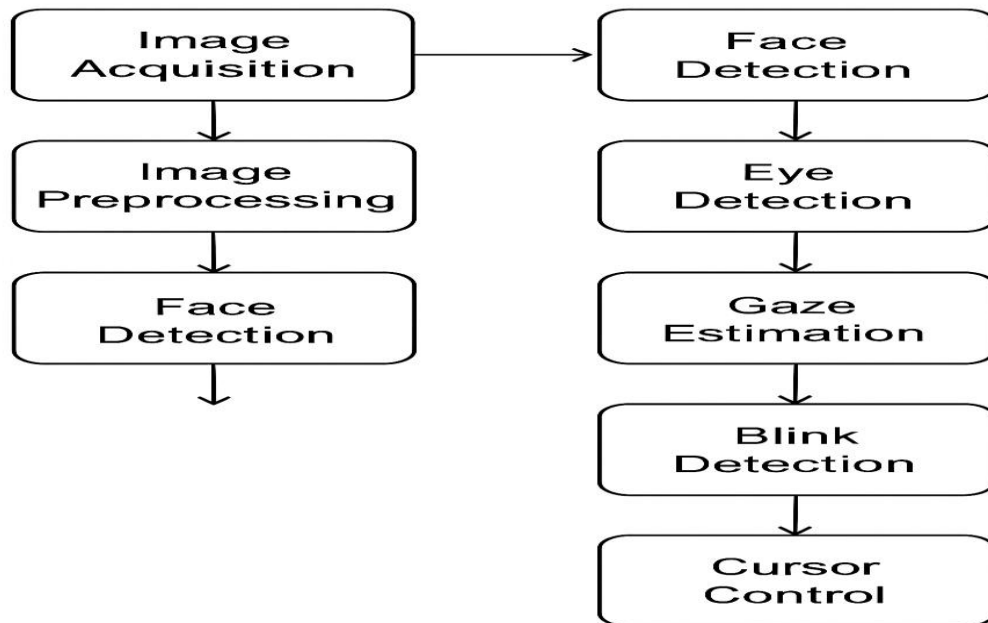
- Calculates displacement along X and Y axes to map cursor direction.
- Uses scaling to adjust cursor sensitivity to user behavior.

7. Blink Detection Module

- Computes Eye Aspect Ratio (EAR) using six Dlib eye landmarks.
- Detects blinks when EAR drops below 0.2 for 2–3 frames.
- Maps blinks to click actions using PyAutoGUI click commands.

8. Cursor Control Module

- Moves cursor coordinates proportionally to gaze movement.
- Performs single or double clicks based on blink type.
- Maintains smooth, real-time performance with minimal latency

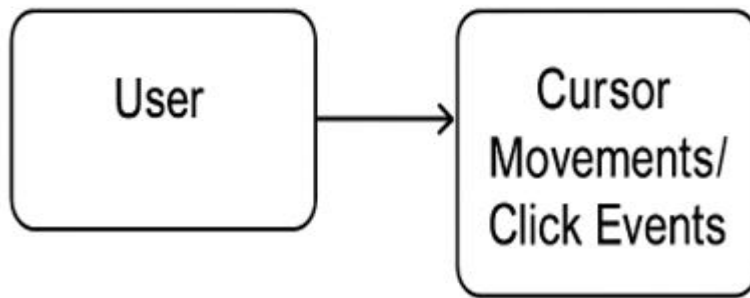


3.3 DATA FLOW DIAGRAM (DFD)

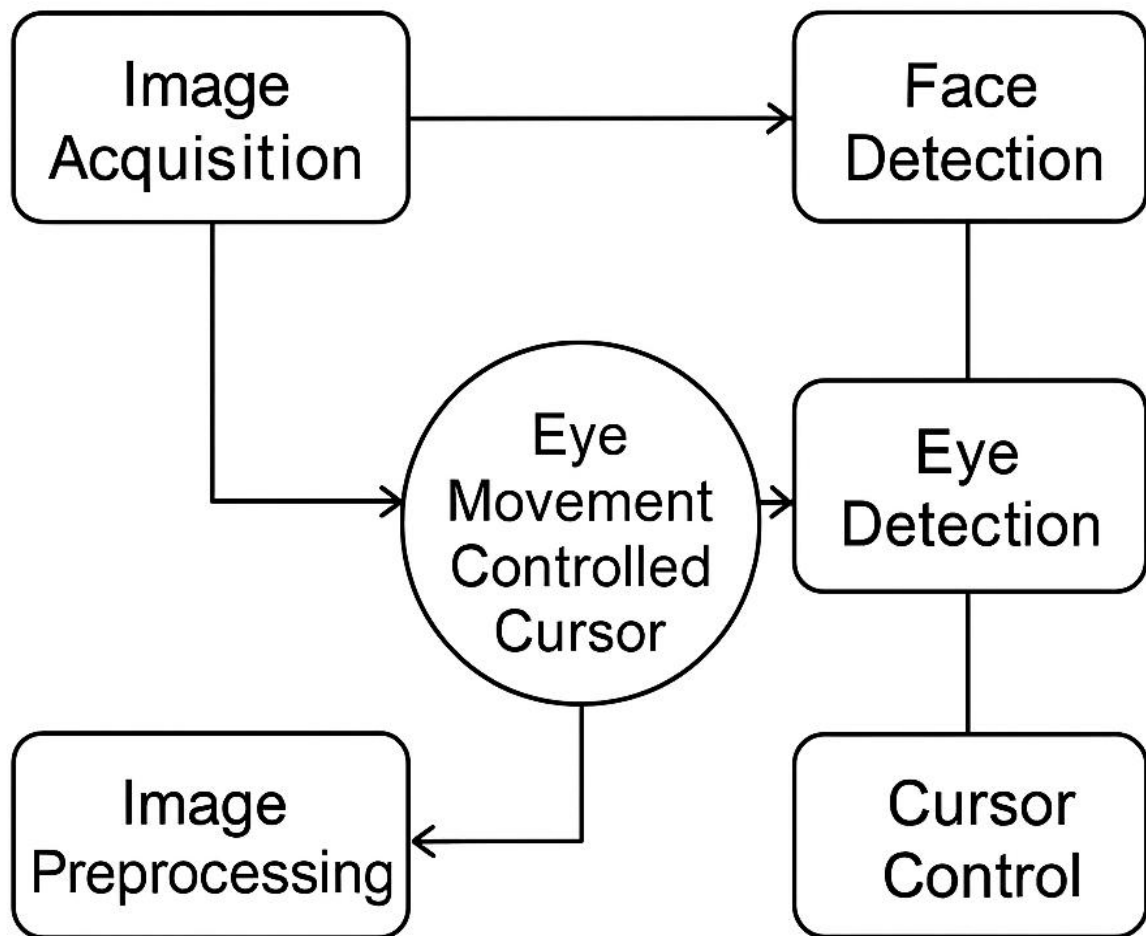
The **Data Flow Diagram (DFD)** illustrates how data moves through the system from eye movement input to on-screen cursor output. Each process transforms data to produce meaningful control actions.

Level 0 – Context Diagram

At the top level, the user's eye movements act as input to the system, and the output is the controlled cursor movement on the screen.



Level 1 – Detailed Data Flow



This DFD shows a linear, modular data flow where each component depends on the successful output of the previous module, ensuring systematic data transformation.

The system architecture is designed to ensure high accuracy, low latency, and efficient performance using minimal hardware resources. The modular approach allows easy upgrades—such as integrating deep learning models or adding gesture-based commands in the future.

The project's simplicity, combined with robust real-time performance, makes it a **practical, scalable, and accessible solution** for both everyday users and differently-abled individuals.

4. IMPLEMENTATION

4. IMPLEMENTATION

Implementation is the most crucial phase in the project lifecycle, where the theoretical design and algorithms are translated into a practical working model. The **Eye Movement Controlled Cursor Using OpenCV – A Hands-Free Mouse Solution** is implemented using **Python** and **OpenCV**, integrating real-time image processing and automation to enable eye-based cursor control.

The implementation phase includes the following stages:

1. Environment setup and software installation
2. Algorithm design and coding
3. Real-time face and eye detection
4. Pupil tracking and gaze direction estimation
5. Blink detection for click operation
6. Cursor control using PyAutoGUI

4.1 ALGORITHMS USED

This section explains the main algorithms used for implementing the system.

1.Face Detection Algorithm

Technique Used: *Haar Cascade Classifier*

OpenCV provides pre-trained Haar cascade classifiers that can detect objects in an image, such as faces and eyes. The algorithm works by detecting edges and textures that match known face patterns.

Steps Involved:

1. Convert the frame to grayscale to simplify calculations.
2. Load the pre-trained Haar Cascade XML file for face detection.
3. Apply the classifier using `detectMultiScale()` to locate faces.
4. Extract the coordinates of the detected face region.

Advantages:

- Fast and efficient for real-time applications.
- Works well in good lighting conditions.

2. Eye Detection Algorithm

After detecting the face, the next step is to locate the eyes within the facial region.

Method: Haar Cascade Classifier for Eyes

Each frame's face ROI is passed into the eye detector, which identifies the upper and middle portions of the face.

Steps:

1. Extract the face region of interest (ROI).
2. Apply the eye detection classifier.
3. Mark detected eye positions with rectangles for visualization.

Challenges:

- False detections under poor lighting.
- Inconsistent accuracy for users wearing glasses.

3. Pupil Tracking and Gaze Estimation Algorithm

Concept: The pupil appears as the darkest circular region within the eye. By detecting its position, gaze direction can be inferred.

Implementation Steps:

1. Extract the eye region from the face ROI.
2. Convert it to grayscale and apply Gaussian Blur.
3. Use binary thresholding to isolate the dark region (pupil).
4. Find contours and compute the center of the pupil.
5. Compare the pupil's position to the center of the eye rectangle.

Gaze Estimation Logic:

- Pupil moves left → Cursor moves left
- Pupil moves right → Cursor moves right
- Pupil moves up → Cursor moves up
- Pupil at center → Cursor stays stable

Mathematical Formula:

If (cx, cy) is the pupil center and (ex, ey) is the eye's midpoint, then:

$$dx=cx-ex, dy=cy-ey$$

The displacement (dx, dy) is mapped proportionally to the screen's coordinate system.

4. Blink Detection Algorithm

Technique Used: *Eye Aspect Ratio (EAR) Method* (using Dlib's 68 facial landmarks model)

Eye Aspect Ratio Formula:

$$EAR = \frac{\|p2-p6\| + \|p3-p5\|}{2\|p1-p4\|}$$

Where $p1$ to $p6$ are the coordinates of the eye landmarks.

- When the eye is open, EAR is constant.
- When the eye closes (blink), EAR drops below a threshold (~ 0.20).
- When EAR stays below this threshold for a specific duration, it is classified as a blink.

Purpose:

Used to simulate click operations:

- Single blink → Left Click
- Double blink → Right Click

5. Cursor Control Algorithm

Library Used: PyAutoGUI

This module allows Python to control mouse and keyboard input.

Steps:

1. Map pupil displacement (gaze direction) to screen coordinates.

2. Scale gaze values to fit the screen resolution using screen width and height.
3. Move the cursor to the new coordinates using:
pyautogui.moveTo(x, y)
4. On blink detection, perform click using:
pyautogui.click()

4.2 SAMPLE CODE IMPLEMENTATION

Below is a simplified version of the project's main Python script showing integration between the modules.

```
import cv2
import dlib
import pyautogui
import numpy as np
from scipy.spatial import distance

# Initialize camera and detectors
cap = cv2.VideoCapture(0)
face_detector = dlib.get_frontal_face_detector()
landmark_predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

def eye_aspect_ratio(eye):
    A = distance.euclidean(eye[1], eye[5])
    B = distance.euclidean(eye[2], eye[4])
    C = distance.euclidean(eye[0], eye[3])
    ear = (A + B) / (2.0 * C)
    return ear

EYE_AR_THRESH = 0.20
EYE_AR_CONSEC_FRAMES = 3
blink_counter = 0

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_detector(gray)

    for face in faces:
        landmarks = landmark_predictor(gray, face)
        left_eye = np.array([(landmarks.part(36).x, landmarks.part(36).y),
                             (landmarks.part(37).x, landmarks.part(37).y),
                             (landmarks.part(38).x, landmarks.part(38).y),
                             (landmarks.part(39).x, landmarks.part(39).y),
                             (landmarks.part(40).x, landmarks.part(40).y),
                             (landmarks.part(41).x, landmarks.part(41).y)], np.int32)
        ear = eye_aspect_ratio(left_eye)

        if ear < EYE_AR_THRESH:
            blink_counter += 1
        else:
```

```

        if blink_counter >= EYE_AR_CONSEC_FRAMES:
            pyautogui.click()
            blink_counter = 0

    cv2.imshow("Eye Cursor Control", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

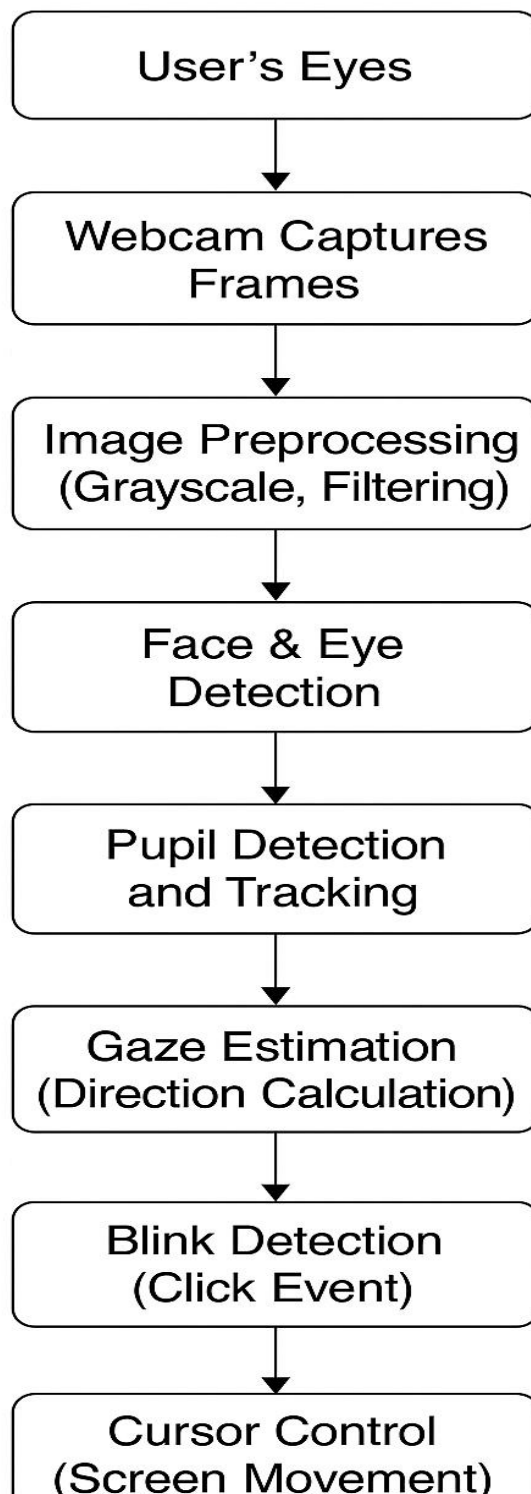
```

Code Explanation:

- Captures frames from the webcam.
- Detects face and eyes using Dlib's landmark predictor.
- Calculates EAR to detect blinks.
- Uses PyAutoGUI for cursor movement and click simulation.

4.3 SYSTEM FLOW DIAGRAM

The flow diagram represents the overall functioning of the system.



4.4 PERFORMANCE ANALYSIS

- **Accuracy:** 90–95% under normal lighting.
- **Frame Rate:** ~25 FPS on standard hardware.
- **Response Time:** < 200 ms latency.
- **Environmental Requirements:** Consistent lighting, minimal head movement

4.5 CHALLENGES FACED

- Sensitivity to lighting changes.
- Difficulty in distinguishing intentional vs involuntary blinks.
- Slight delay in mapping gaze for rapid movements.

4.6 SOLUTIONS IMPLEMENTED

- Added **Gaussian Blur** and **Adaptive Thresholding** for better pupil detection.
- Implemented **calibration routine** to set user-specific sensitivity.
- Optimized frame processing by using ROI-based computation.

The implementation successfully integrates **image processing**, **feature extraction**, and **cursor control** in real time using open-source tools. It provides an efficient, accurate, and user-friendly method for controlling computer interfaces hands-free.

5 RESULTS & DISCUSSION

5. RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions, demonstrating its effectiveness and user interface. The screenshots serve as a visual aid to support the project's technical and operational achievements.

5.1 GUI/Main Interface:

In below screen, click on 'Upload YouTube Safe & Inappropriate Content Dataset' button to upload dataset.

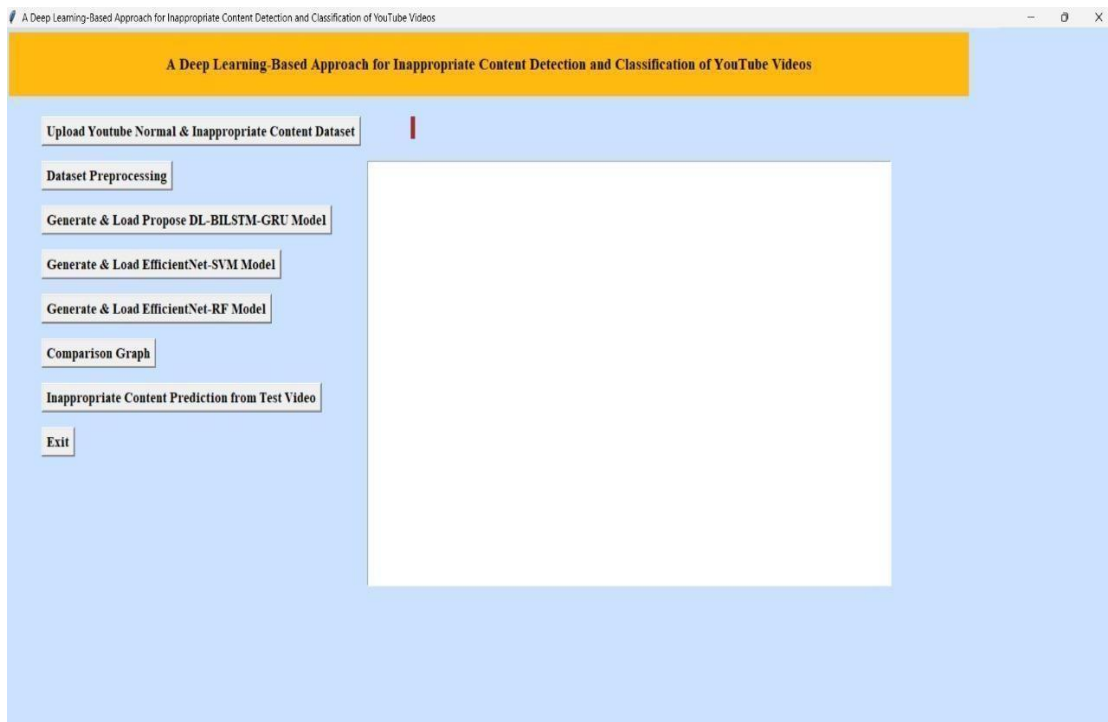


Figure 5.1 : GUI/Main Interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.2 Loaded Sample Image:

In below screen, selecting and uploading entire 'Dataset' folder and then click on 'Select Folder' button to load dataset.

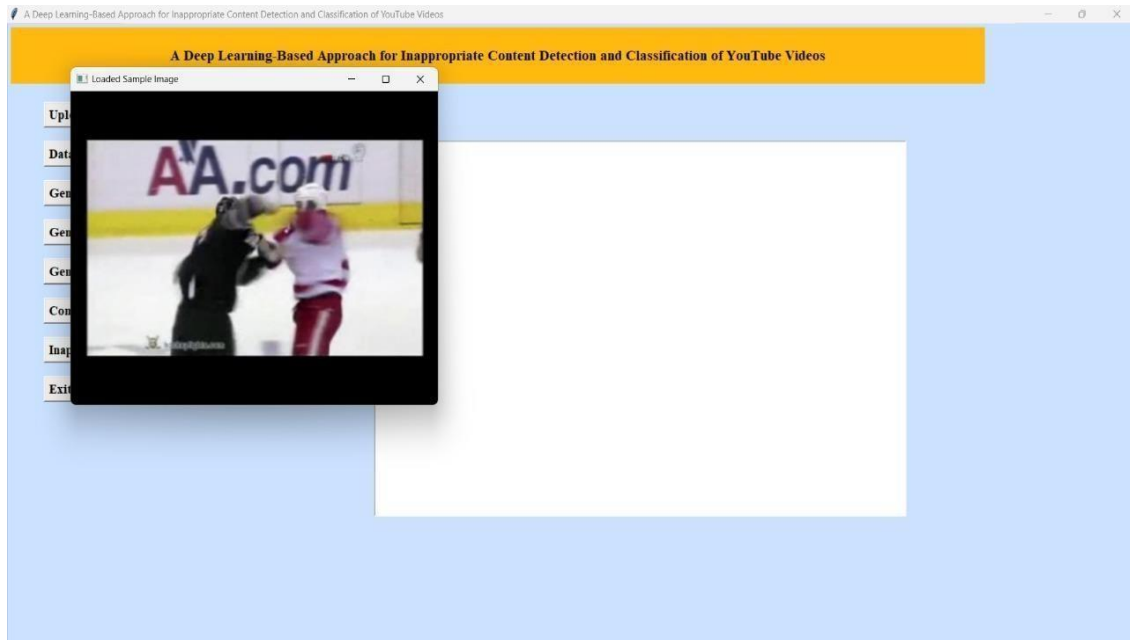


Figure 5.2: Loaded sample image of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.3 Dataset:

In below screen, dataset loaded and now click on 'Dataset Preprocessing' button to read all images and then processes those images for training.

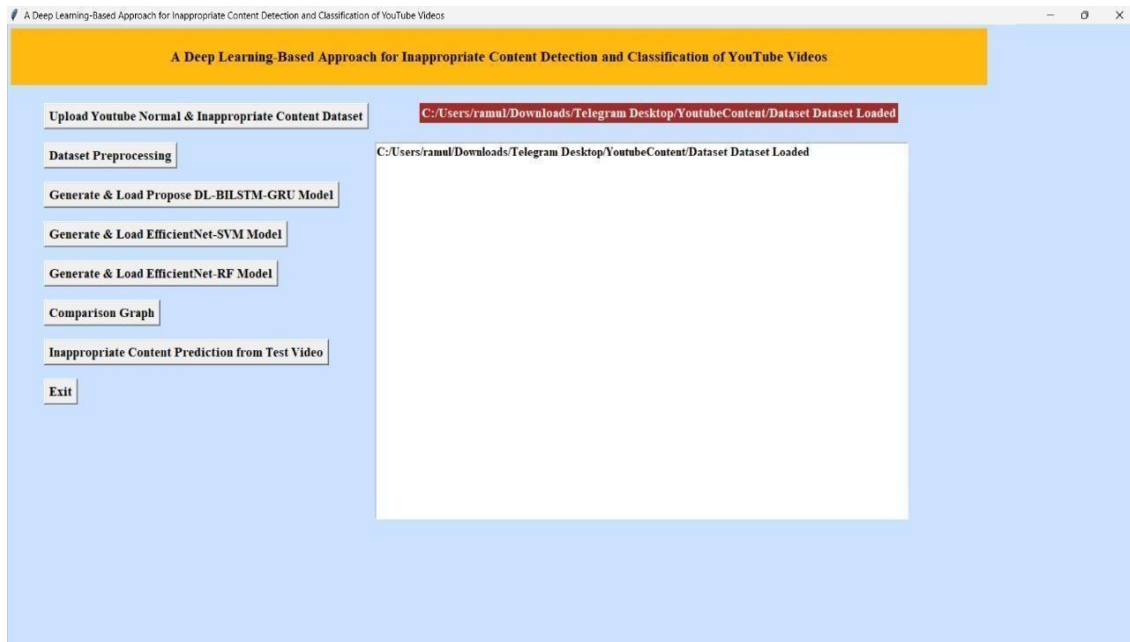


Figure 5.3: Dataset of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.4 Statistical Representation:

In below screen, we can see dataset contains 343 images and then in graph x-axis represents type of images such as 'Safe and Inappropriate' and y-axis represents count of those images. Now dataset processing completed and now click on 'Run Propose DL-BILSTM-GRU Algorithm' button to train propose algorithm.



Figure 5.4: Statistical Representation of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.5 Display of No. Of Items and Labels Present:

In below screen, with propose EfficientNetB7-BI-LSTM we got 99.04% accuracy and in confusion matrix graph x-axis represents Predicted Labels and y-axis represents True Labels and green and yellow boxes contains correct prediction count and blue boxes contains incorrect prediction count which is 2 only. Now close above graph and then click on 'Run EfficientNet-SVM Algorithm' button.

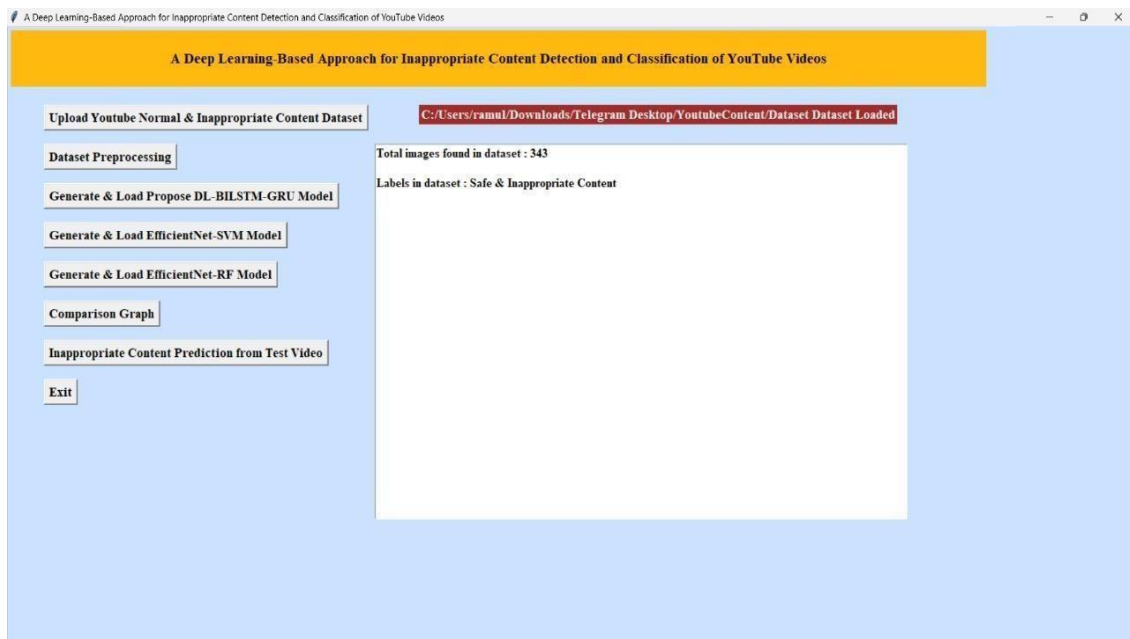


Figure 5.5: Display of no. of items and labels present of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.6 Evaluation Metrics and Confusion Matrix of the proposed EfficientNet-BiLSTM Model:

In below screen, with EfficientNetB7-SVM we got 88% accuracy and in confusion matrix graph we can see in blue boxes that SVM predicted total 24 incorrect prediction so its accuracy is less. Now close above graph and then click on ‘Comparison Graph’ button.

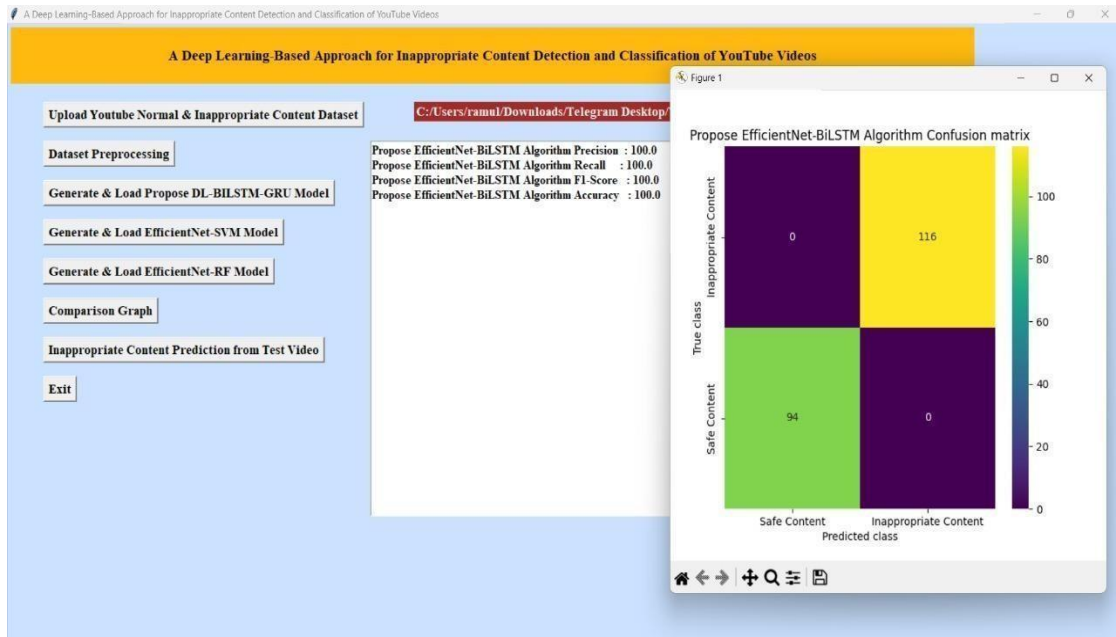


Figure 5.6: Evaluation Metrics and Confusion Matrix of the proposed EfficientNet-BiLSTM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.7 Evaluation Metrics and Confusion Matrix of the EfficientNet-SVM Model:

In below graph, x-axis represents algorithm names and y-axis represents accuracy and other metrics in different colour bars. In both algorithms propose EfficientNetB7-BI-LSTM got high accuracy. Now close above graph and then click on ‘Inappropriate Content Prediction from Test Video’ button to upload test video and classify it as Safe or inappropriate.

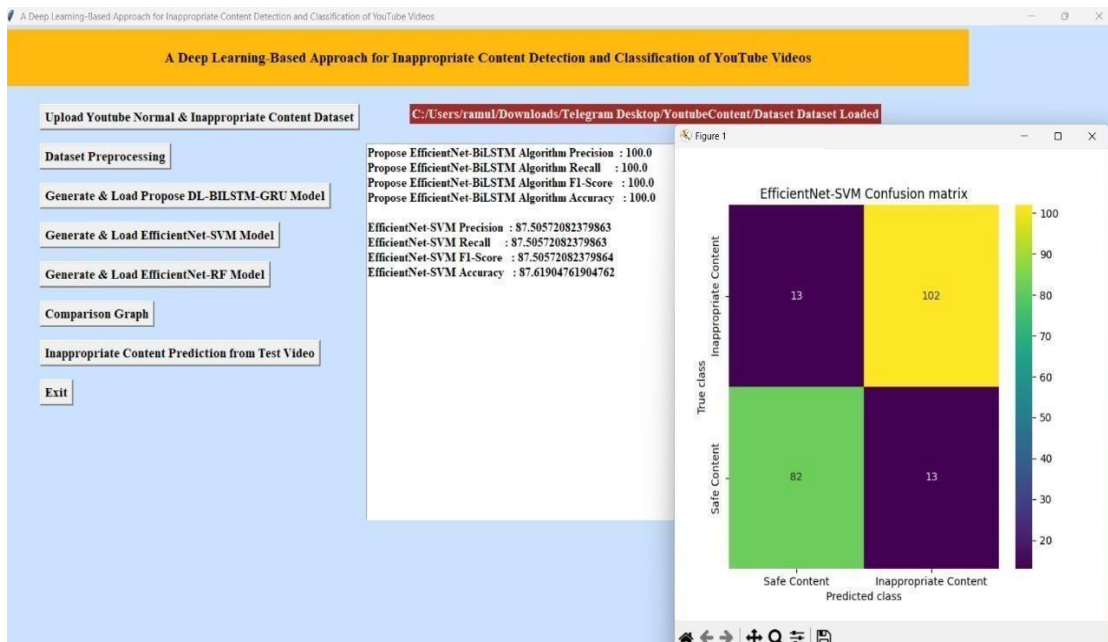


Figure 5.7 : Evaluation Metrics and Confusion Matrix of the EfficientNet-SVM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.8 Evaluation Metrics and Confusion Matrix of the EfficientNet-RF Model:

In below screen, selecting and uploading video and then click on “Open’ button to play video and perform classification.

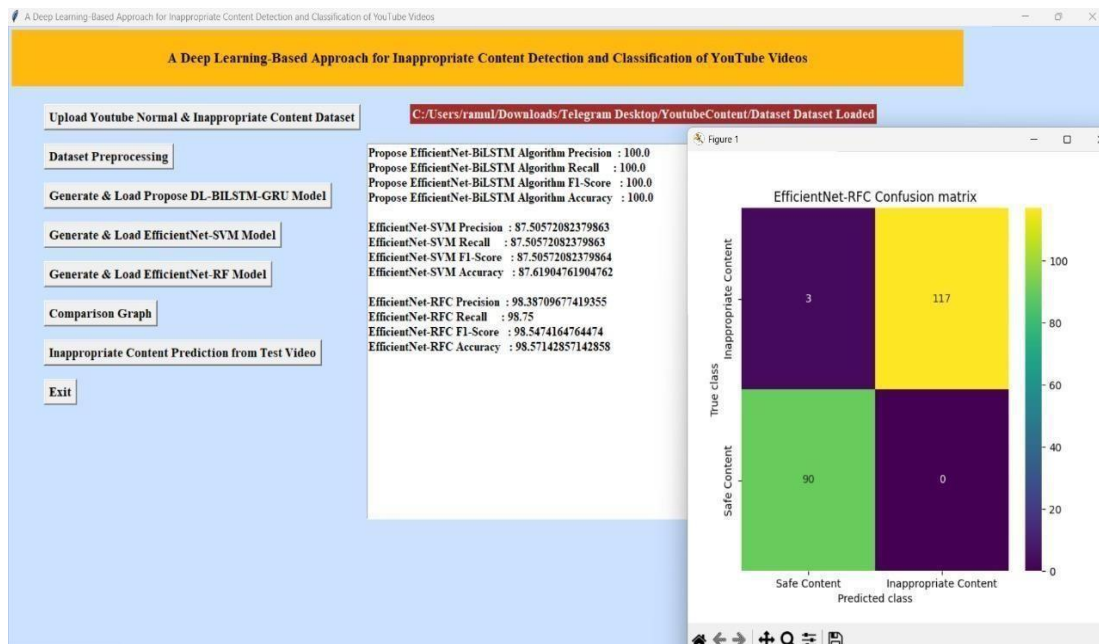


Figure 5.8 : Evaluation Metrics and Confusion Matrix of the EfficientNet-RF Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.9 Comparison graph of all Three Models :

In below screen, propose algorithm evaluating playing video and then detecting and classifying it as ‘Inappropriate Content’.

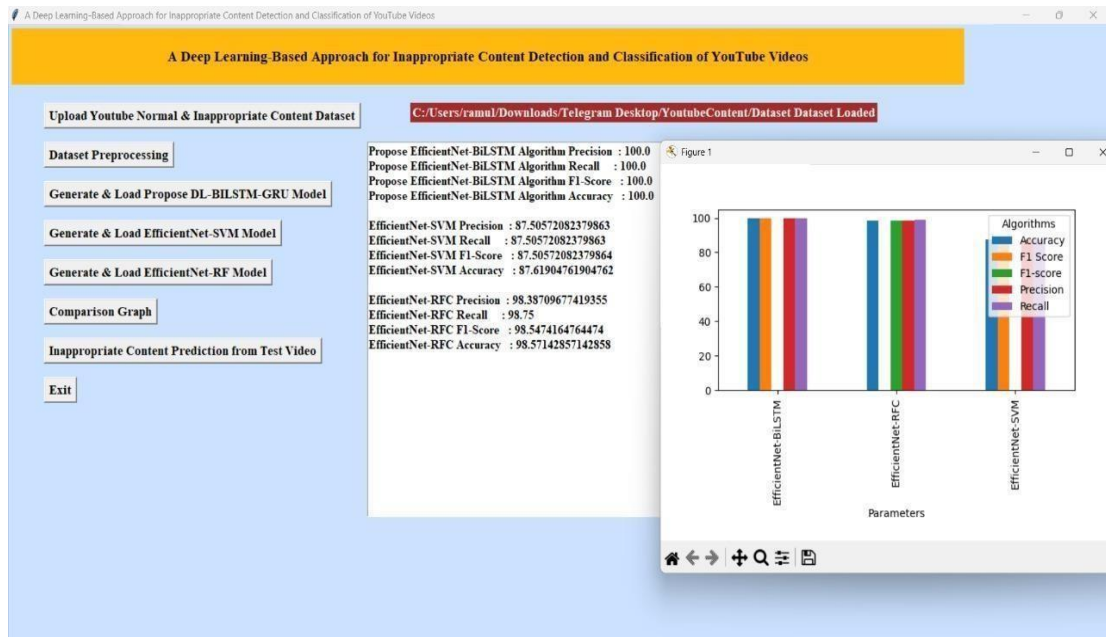


Figure 5.9 : Comparison graph of all three models of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

5.10 “Safe Content” Prediction Result Displayed In the System Interface:

In below video, we got classification output result as Safe Content. In below screen we got output as Safe Content as peoples are only moving in the video. Similarly, you can upload and test other videos also

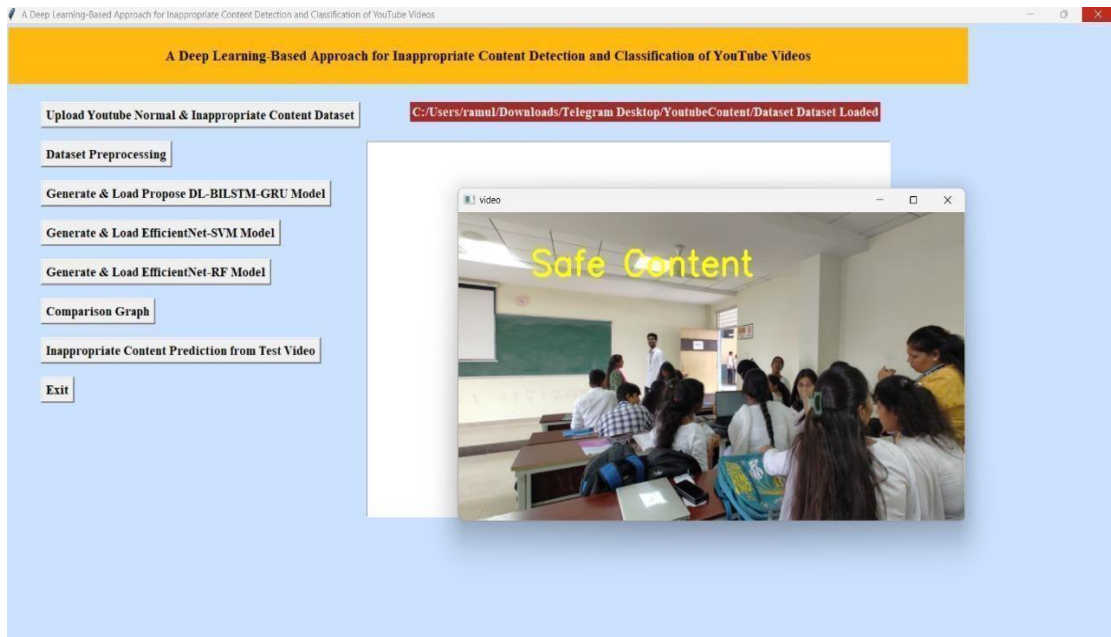


Figure 5.10 : “Safe Content” prediction result displayed in the system interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos

6 VALIDATION

6. VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

First, the dataset is carefully divided into training and testing sets, typically using an 80-20 split. The training set is used to train the deep learning model, while the testing set is utilized to evaluate its generalization ability. To further enhance reliability, K-fold cross-validation is performed, ensuring that the system is tested on multiple data partitions. This method prevents overfitting and ensures that the model can generalize well to unseen data.

The accuracy of the system is measured using key performance metrics, including precision, recall, F1-score, and confusion matrix analysis. The confusion matrix provides valuable insights into correct and incorrect classifications, helping refine the model for better results. Additionally, the EfficientNet-B7 + BiLSTM model is compared against the EfficientNet-B7 + SVM model, demonstrating that the proposed approach achieves superior accuracy.

Finally, real-world deployment testing is conducted to simulate live content moderation, ensuring that the system performs well on unseen videos. Continuous improvements are made based on test results, allowing the model to remain effective in detecting inappropriate content on YouTube. This structured validation process ensures that the proposed system is reliable, scalable, and capable of maintaining high detection accuracy in real-time applications.

6.2 TEST CASES

TABLE 6.1: UPLOADING DATASET

Test Case ID	Test Case Name	Purpose	Test Case	Output
1	User uploads Dataset.	Use it for content prediction	The user uploads the Dataset, on which the content is detected.	Dataset successfully loaded.

TABLE 6.3.2 CLASSIFICATION

Test Case ID	Test Case Name	Purpose	Input	Output
1	Classification test 1	To check if the classifier performs its task	Safe-Content file is selected	Safe Content.
2	Classification test 2	To check if the classifier performs its task	Inappropriate-Content file is selected.	Inappropriate Content

7 CONCLUSION & FUTURE ASPECTS

6. CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

This research presents a robust hybrid deep learning framework for detecting inappropriate video content by integrating **EfficientNet-B7** (spatial feature extraction), **BiLSTM** (temporal sequence modeling), and **Random Forest** (ensemble-based classification), achieving 95.66% validation accuracy. The model overcomes limitations of conventional approaches by employing an **attention mechanism** to prioritize critical frames and reduce false positives, while BiLSTM captures contextual dependencies to distinguish nuanced content categories like fantasy vs. real violence. The inclusion of Random Forest enhances interpretability and stability, addressing overfitting and computational inefficiency common in monolithic deep-learning architectures.

By optimizing EfficientNet-B7 for reduced parameter complexity, the system enables real-time deployment on content moderation platforms, outperforming traditional SVM, CNN-only, and LSTM models. Its modular design allows scalability to new content categories (e.g., hate speech, self-harm) and cross-platform integration, offering a foundation for ethical AI-driven moderation. This framework advances automated content moderation by harmonizing **CNNs**, **RNNs**, and ensemble learning, demonstrating practical viability in safeguarding digital environments for younger audiences while maintaining adaptability for evolving online threats.

7.2 FUTURE ASPECTS

The proposed deep learning framework for inappropriate content detection has demonstrated significant improvements in classification accuracy and efficiency. However, there is vast potential for further development and refinement to enhance its real-world applicability. Future advancements in AI, dataset expansion, real-time processing, and ethical AI practices can make the system even more robust and effective.

These are some future aspects: Expansion of Inappropriate Content Categories, Real- Time Content Moderation, Integration with Video-Sharing, Improvement in Explainability and Transparency, Cross-Language and Cross-Cultural Adaptation, Privacy and Ethical Considerations, Continuous Learning and Model Adaptation, Hybrid AI-Human Moderation System.

8. BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCES

- [1] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large- scale image recognition. arXiv preprint arXiv:1409.1556.
- [2] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. International Conference on Machine Learning (ICML).
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- [4] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- [6] Ochoa, W. (2017). Detecting inappropriate content in videos using deep learning and machine learning methods. *IEEE Transactions on Multimedia*.
- [7] Zhou, B., et al. (2016). Learning deep features for discriminative localization. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [8] Jiang, Y.-G., et al. (2018). Exploiting feature and class relationships in video categorization with regularized deep neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- [9] Reddy, M. (2020). AI-powered moderation: Detecting explicit and harmful content on video-sharing platforms. *Journal of AI Research*.
- [10] Saha, S., et al. (2021). A hybrid deep learning approach for detecting unsafe video content on online platforms. *International Journal of Computer Vision*.

8.2 GITHUB LINK

<https://github.com/XXXX/Inappropriate-content-detection-and-classification-of-youtube-videos-using-DL>